

Эффективность на практике

Оптимизация инференса

Алексей Шамшиев



education

Содержание

Вспомнить всё	3
KV-кэширование	6
спекулятивный декодинг	11
vLLM и paged attention	19
gguf и ray	27

Вспомнить
всё



Recap: Self-attention

$\tilde{E} \in \mathbb{R}^{n \times d}$ — матрица эмбеддингов, n — кол-во слов, d — внутренняя размерность

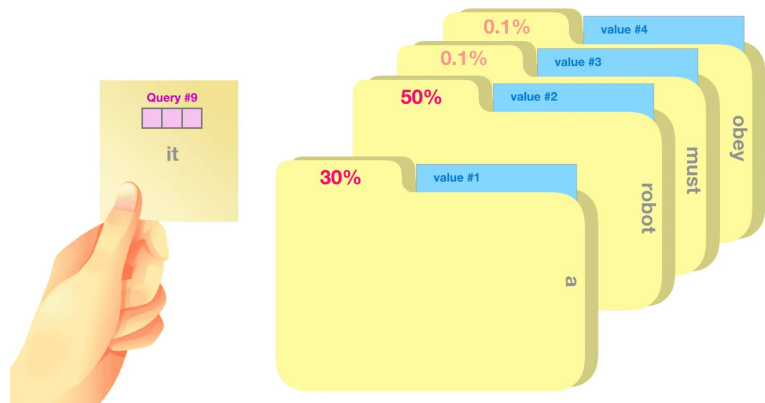
Даны матрицы $Q, K, V \in \mathbb{R}^{d \times d}$

$$\tilde{Q} = \tilde{E} \times Q, \tilde{K} = \tilde{E} \times K, \tilde{V} = \tilde{E} \times V$$

Attention для i -ого слова будет считаться как:

$$\alpha = \text{softmax}(\tilde{K} \times \widetilde{Q_i^T}) \in \mathbb{R}^n$$

$$h_i = \sum_{j=1}^n \alpha_j * \tilde{V}_j$$

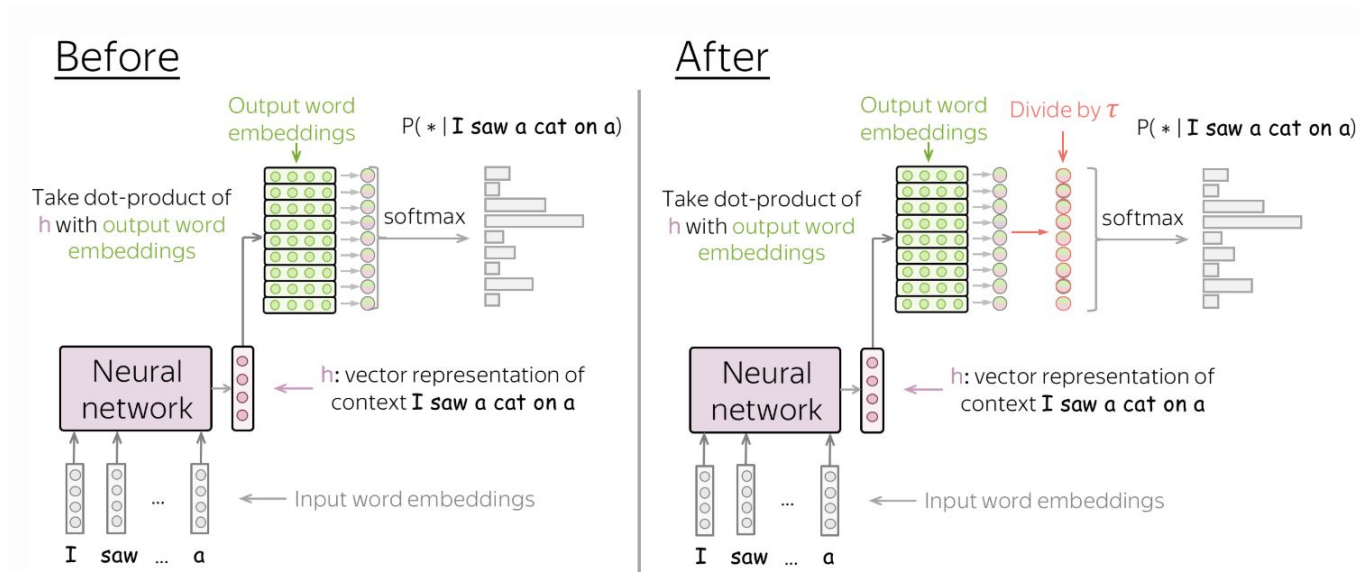


картинка взята [отсюда](#)

Recap: generation with temperature

Coherence and Diversity tradeoff

Температура позволяет
балансировать между
разнообразием и качеством
генерации.



Formally, the computations change as follows:

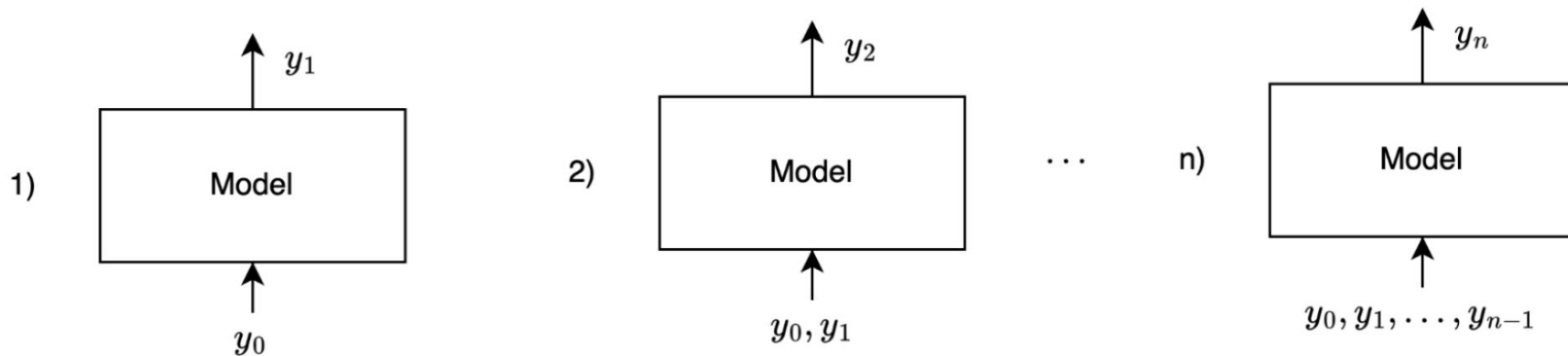
$$\frac{\exp(h^T w)}{\sum_{w_i \in V} \exp(h^T w_i)} \rightarrow \frac{\exp\left(\frac{h^T w}{\tau}\right)}{\sum_{w_i \in V} \exp\left(\frac{h^T w_i}{\tau}\right)}$$

τ - softmax temperature

kv-кэширование



Наивная генерация



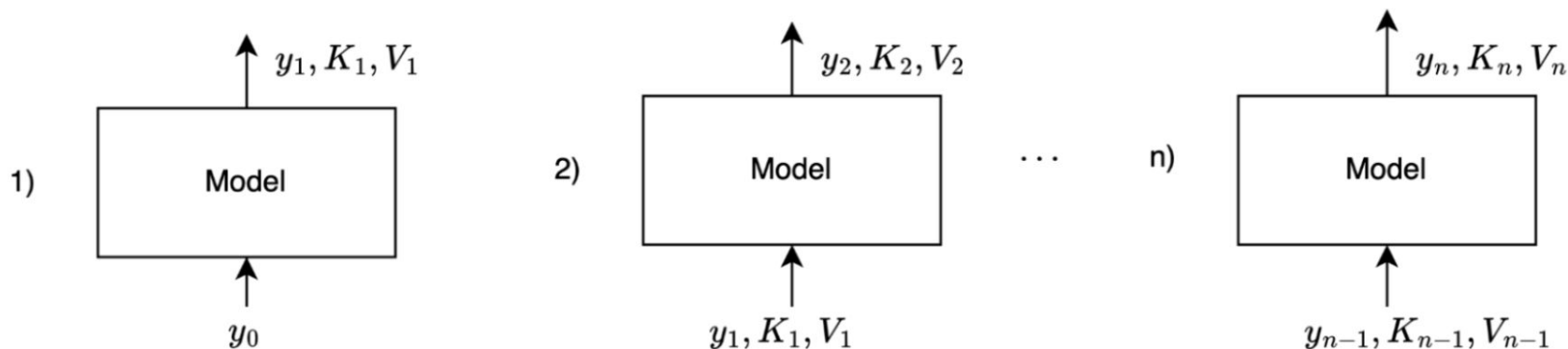
На очередном шаге прогоняем через модель всю уже сгенерированную последовательность, чтобы сгенерировать следующий токен.

Время работы всей генерации: $O(n^3)$

kv-кэширование

Внутреннее представление уже сгенерированных токенов не меняется при добавлении новых токенов справа (ведь токены не могут "смотреть вперед"), поэтому можно их не считать каждый раз заново, а запомнить.

Чтобы получить представление очередного добавленного токена, нужно знать ключи и значения всех предыдущих, поэтому будем их запоминать (key-value cache).



Время работы всей генерации: $O(n^2)$

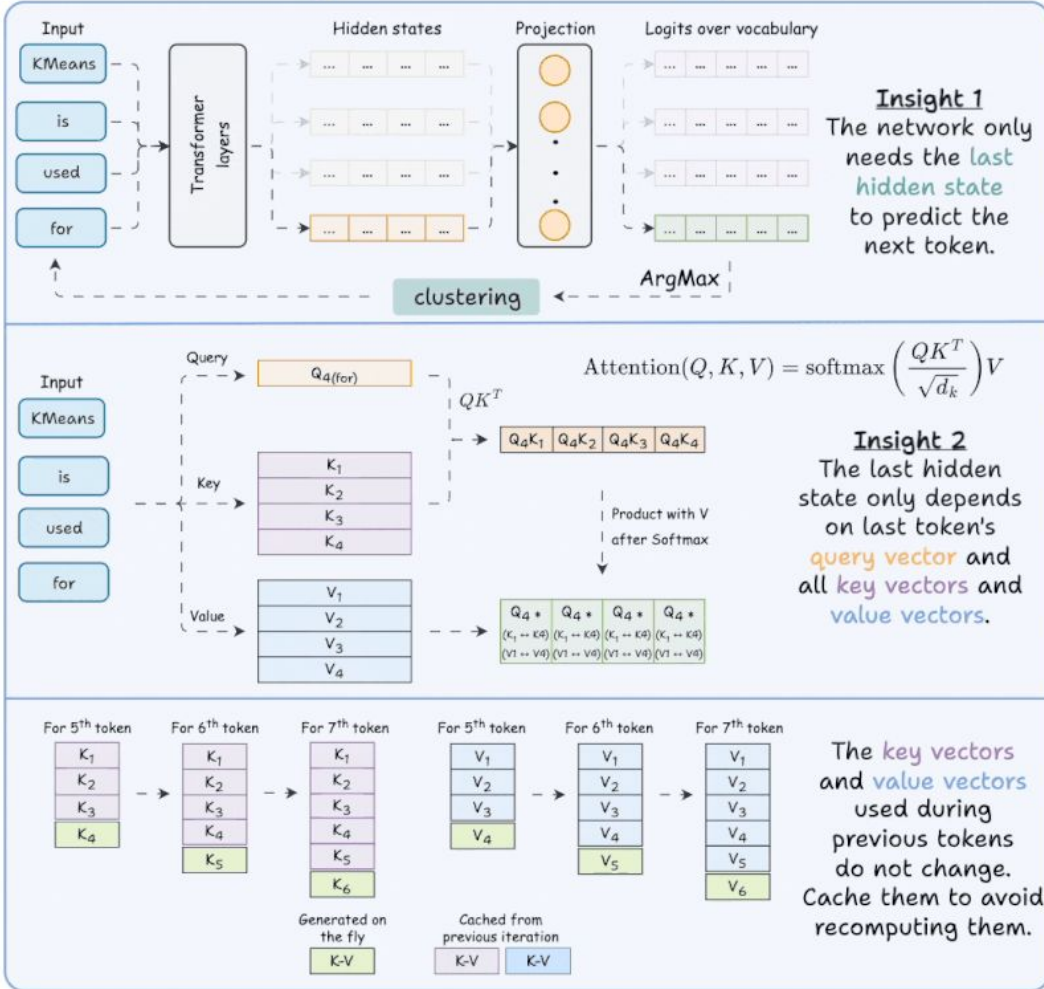
Аналогия с cross-attention

- cross-attention хранит key/value для контекста — KV-cache делает то же самое, но для self-attention
- Вместо вычисления self-attention с нуля, просто добавляем новые KV-пары.

картинка взята [отсюда](#)

KV Caching in LLMs

join.DailyDoseofDS.com



Итого

- Запоминаем, а не пересчитываем каждый раз keys и values
- Ускоряем генерацию на инференсе

Фактор	Без kv-cache	С kv-cache
Время генерации	$O(N^3d)$	$O(N^2d)$
Память	$O(N*d)$	$O(N*d)$

Attention Mechanism	KV Cache per Token (# Element)	Capability
Multi-Head Attention (MHA)	$2n_h d_h l$	Strong
Grouped-Query Attention (GQA)	$2n_g d_h l$	Moderate
Multi-Query Attention (MQA)	$2d_h l$	Weak
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	Stronger

Table 1 | Comparison of the KV cache per token among different attention mechanisms. n_h denotes the number of attention heads, d_h denotes the dimension per attention head, l denotes the number of layers, n_g denotes the number of groups in GQA, and d_c and d_h^R denote the KV compression dimension and the per-head dimension of the decoupled queries and key in MLA, respectively. The amount of KV cache is measured by the number of elements, regardless of the storage precision. For DeepSeek-V2, d_c is set to $4d_h$ and d_h^R is set to $\frac{d_h}{2}$. So, its KV cache is equal to GQA with only 2.25 groups, but its performance is stronger than MHA.

спекулятивный декодинг

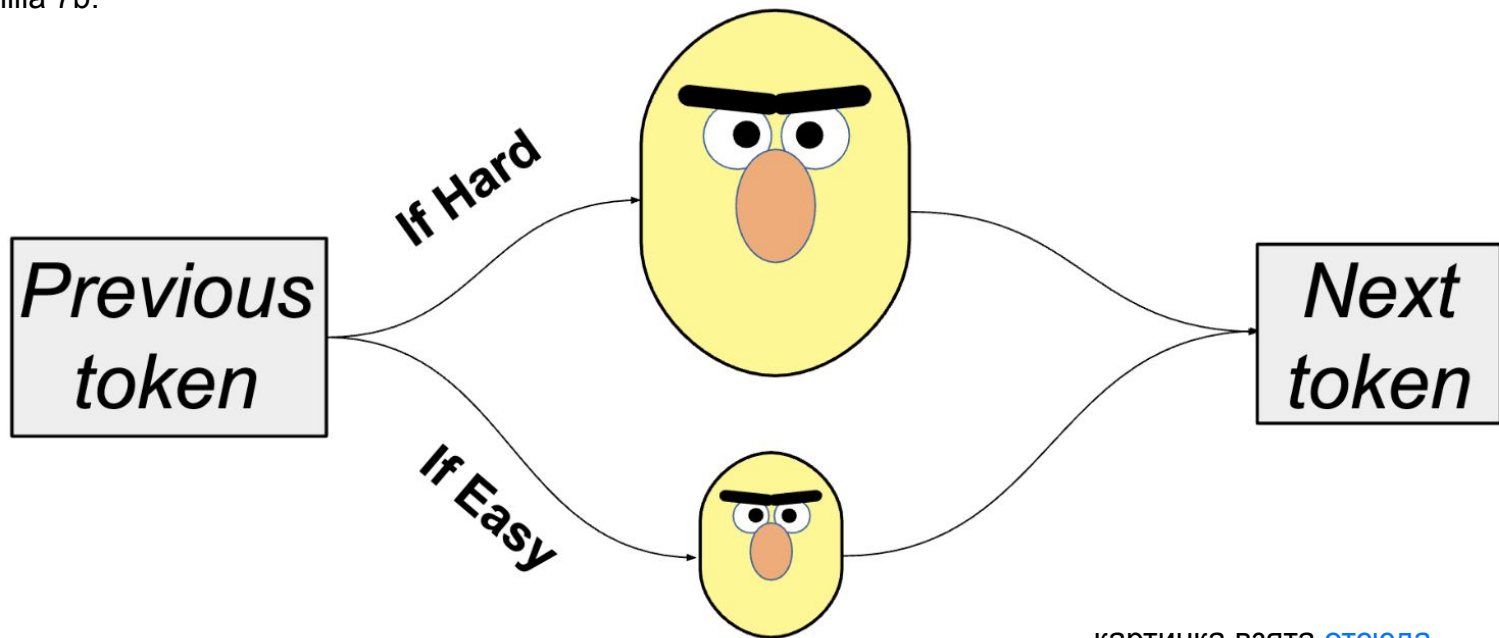


Идея

Большая модель - target (q) исходная модель

Маленькая модель - draft-модель (p),
имеющая размеры в 10-100 раз меньше q.

Авторы из deermind использовали
Chinchilla 70b и Chinchilla 7b.

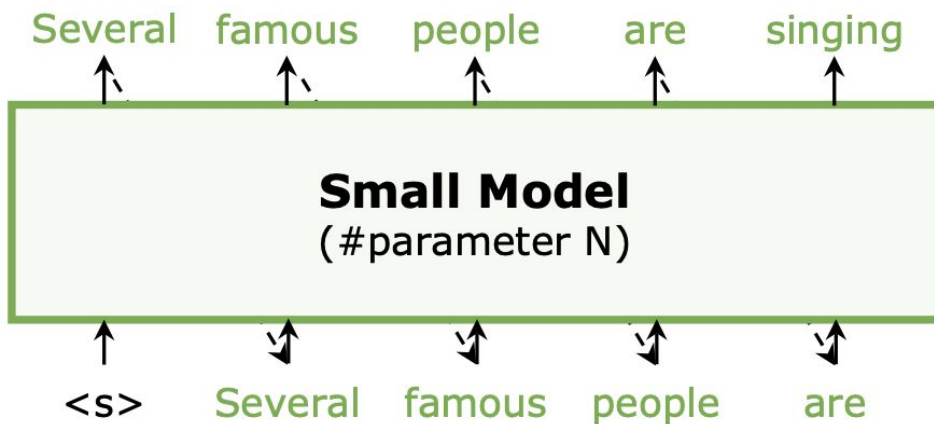


Speculative Decoding

arXiv:2211.17192v2

Greedy decoding:

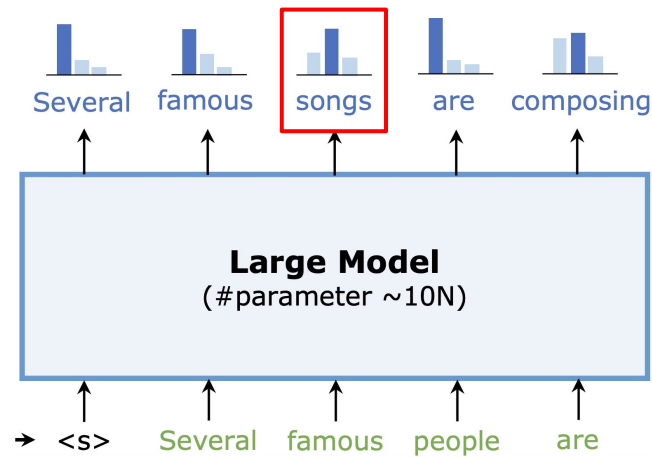
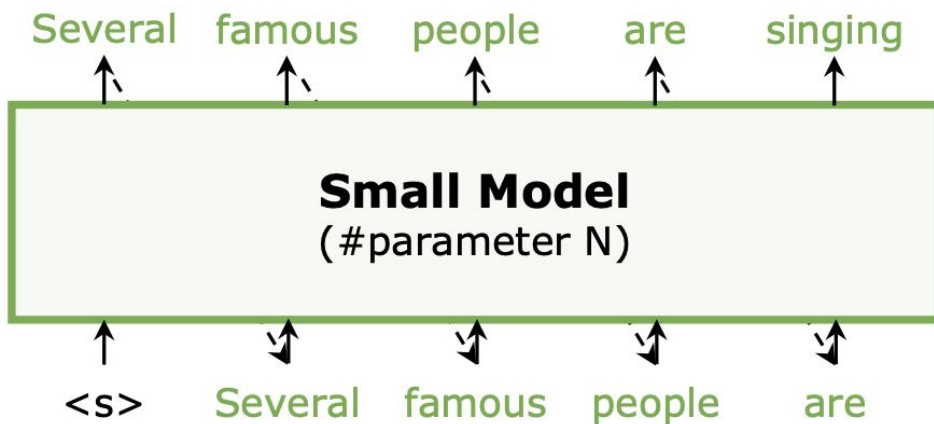
- Шаг 1: генерация draft-моделью



Speculative Decoding

Greedy decoding:

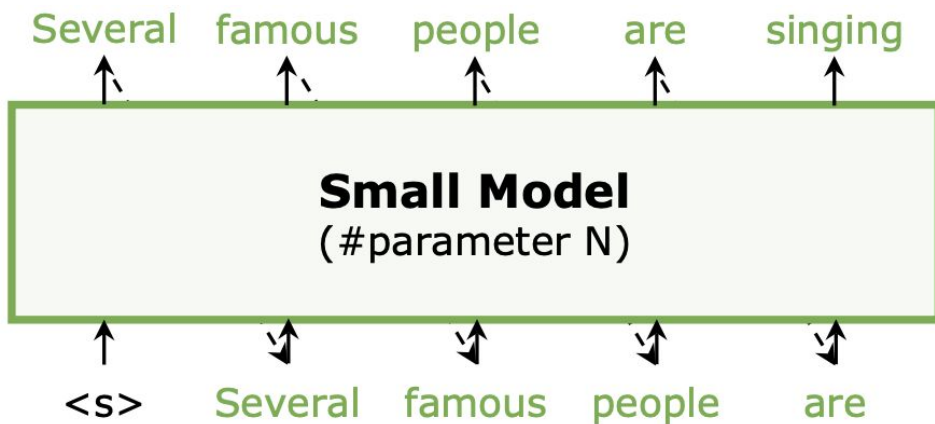
- Шаг 1: генерация draft-моделью
- Шаг 2: Проверка исходной моделью (параллельно)



Speculative Decoding

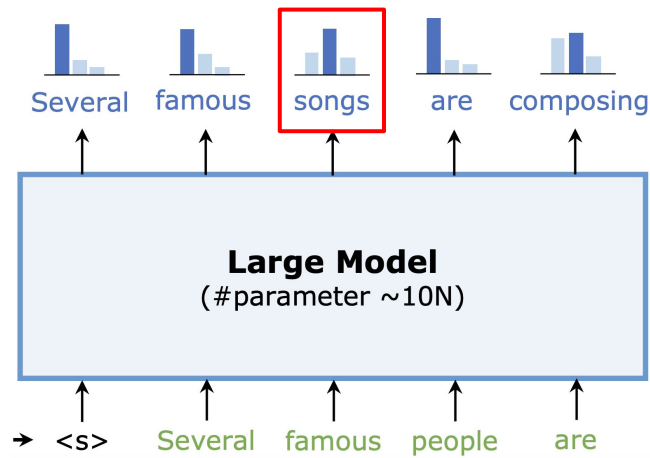
Greedy decoding:

- Шаг 1: генерация draft-моделью
- Шаг 2: Проверка исходной моделью (параллельно)
 - Принимаем часть токенов draft-модели
 - Принимаем один токен target-модели



Сгенерированный текст:

Several famous songs

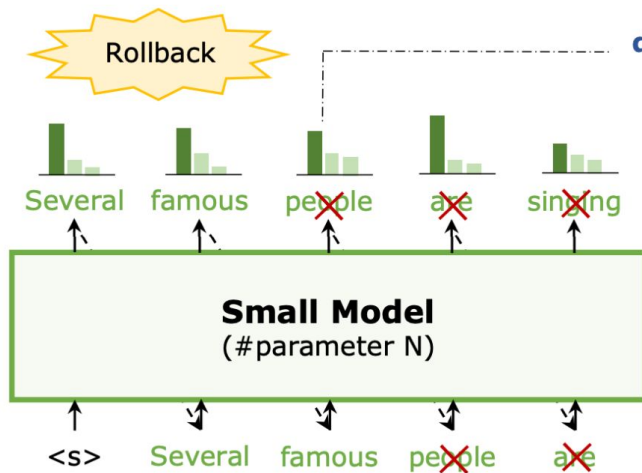


Speculative Decoding

Greedy decoding:

- Шаг 1: генерация draft-моделью
- Шаг 2: Проверка исходной моделью (параллельно)
 - Принимаем часть токенов draft-модели
 - Принимаем один токен target-модели

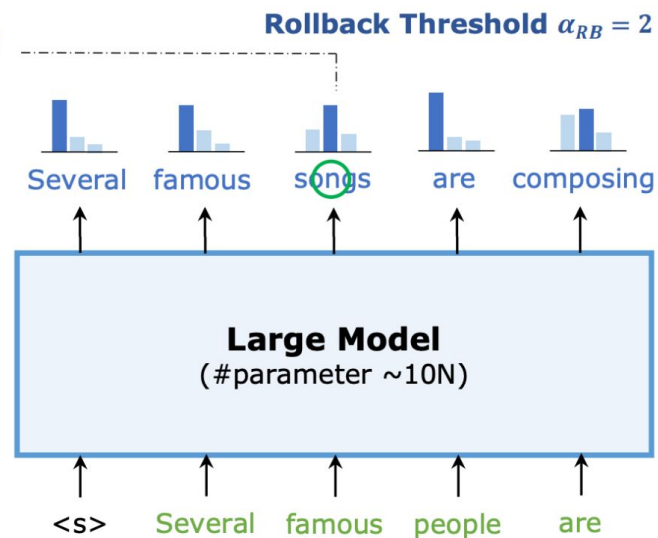
Повторяем



Generated Text: **Several** famous songs

Сгенерированный текст:

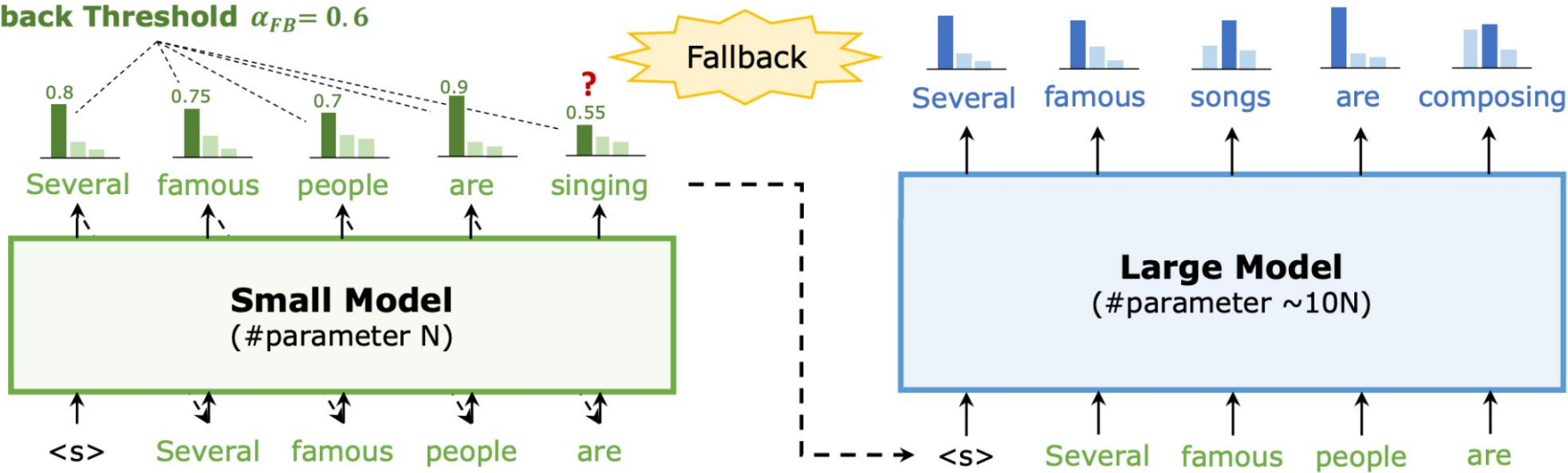
Several famous songs



Speculative Decoding

Sampling (temperature, top-p, top-k): Генерируем токены маленькой моделью и обращаемся к большой, если не уверены.

Fallback Threshold $\alpha_{FB} = 0.6$



Результаты авторов

Table 1 | **Chinchilla performance and speed on XSum and HumanEval with naive and speculative sampling at batch size 1 and $K = 4$.** XSum was executed with nucleus parameter $p = 0.8$, and HumanEval with $p = 0.95$ and temperature 0.8.

Sampling Method	Benchmark	Result	Mean Token Time	Speed Up
ArS (Nucleus)	XSum (ROUGE-2)	0.112	14.1ms/Token	1×
SpS (Nucleus)		0.114	7.52ms/Token	1.92×
ArS (Greedy)	XSum (ROUGE-2)	0.157	14.1ms/Token	1×
SpS (Greedy)		0.156	7.00ms/Token	2.01×
ArS (Nucleus)	HumanEval (100 Shot)	45.1%	14.1ms/Token	1×
SpS (Nucleus)		47.0%	5.73ms/Token	2.46×

Vllm и paged attention



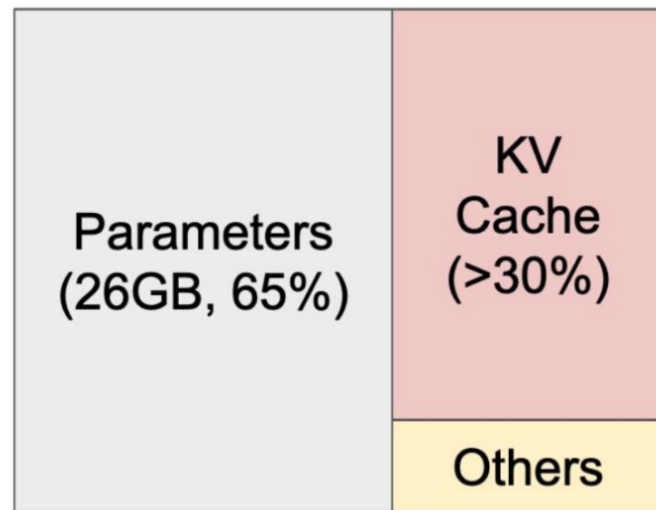
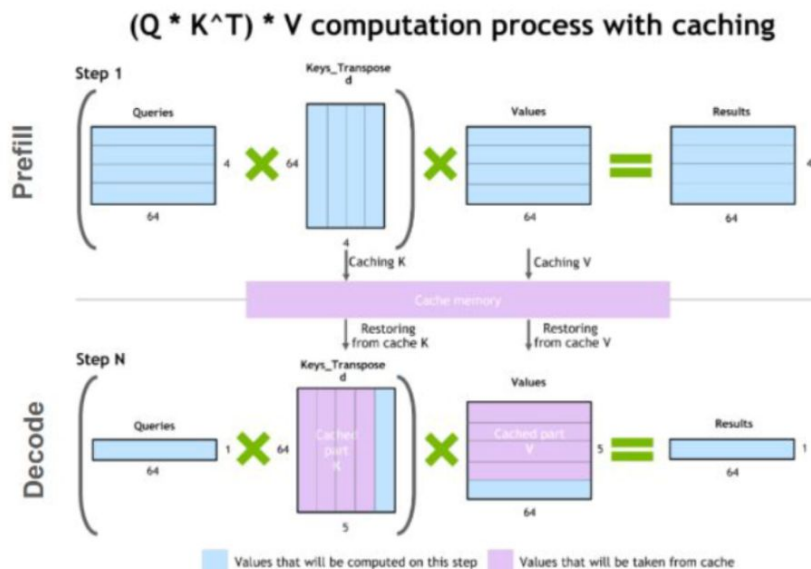
vllm: easy, fast, and cheap LLM serving for everyone

- Лучшее использование GPU-памяти → больше потоков (batch) на одной карте
- Оптимизированный стриминг токенов
- Увеличенная скорость генерации



Что внутри Vllm?

Answer: KV Cache



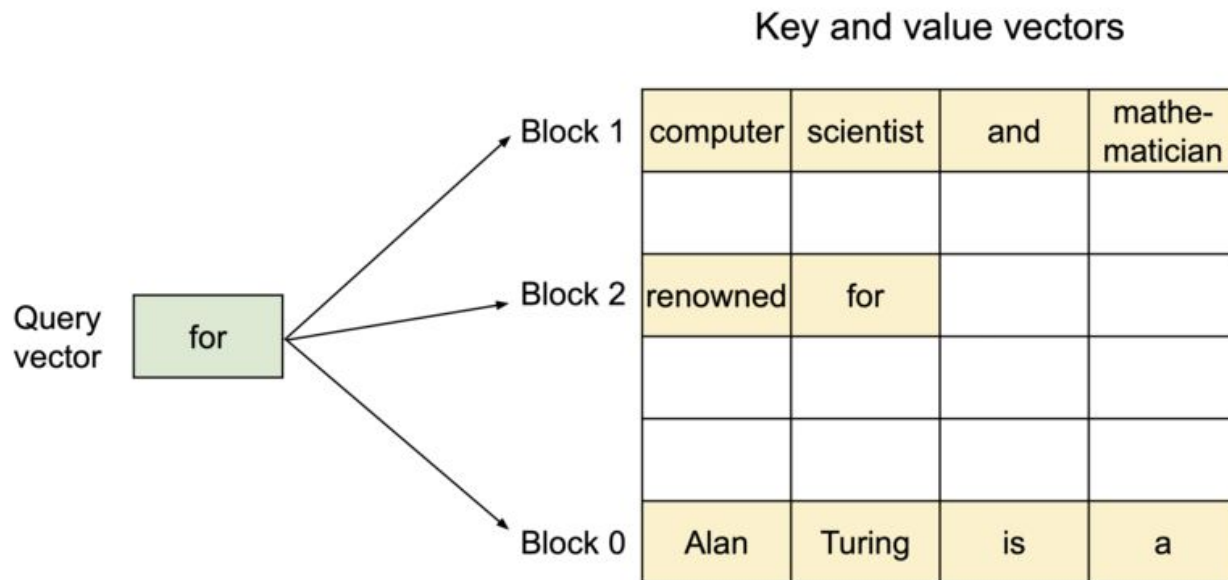
NVIDIA A100 40GB

Image: arxiv.org/pdf/2309.06180

Paged attention

arXiv:2309.06180v1

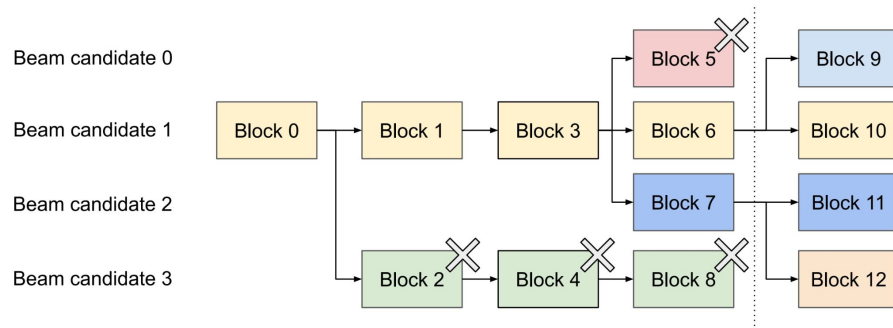
- Разбиваем KV cache на страницы (аналог виртуальной памяти)
- Храним только активные токены
- Избегаем переполнения кэша



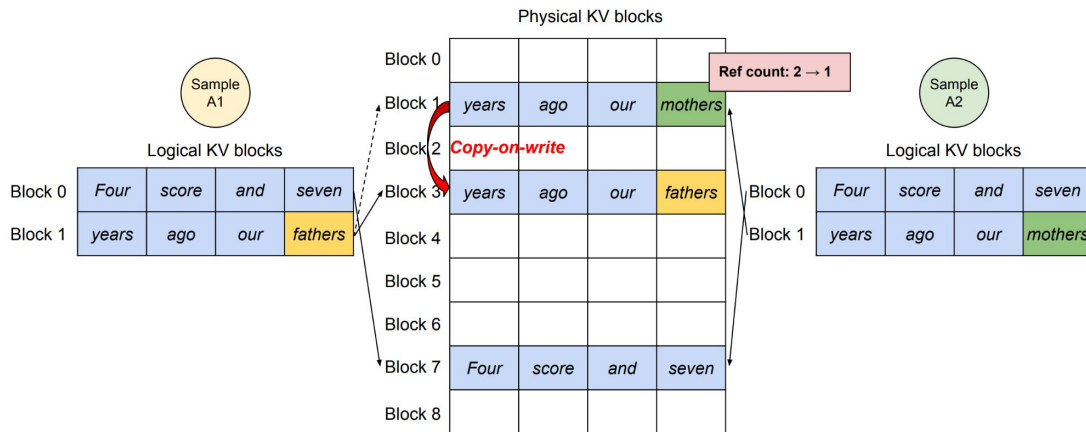
Эффективность Paged attention

Можем переиспользовать KV-cache

- При одинаковых префиксах у разных запросов (можно запомнить attention для промпта)
- Не пересчитываем каждый раз attention при кандидатах в beam search
- Используем меньше памяти при генерации



Пример использования Beam Search



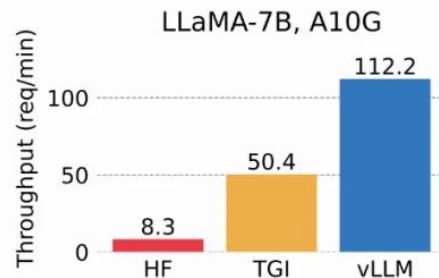
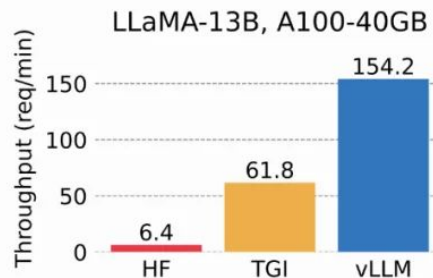
Аналоги

	vLLM	Llama.cpp	TGI
основное назначение	Оптимизированный GPU-инференс	Легковесный движок для CPU/GPU	Production-ready сервер для LLM
PagedAttention	+	-	+
Оптимизация KV-cache	+	-	+
Поддержка квантованных моделей	+ -	+	-
Поддержка стриминга токенов	+	+ -	+

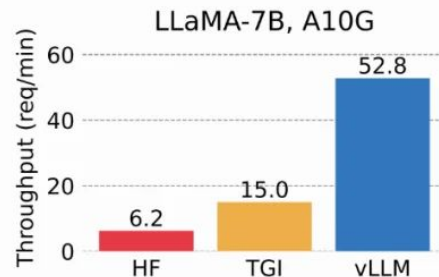
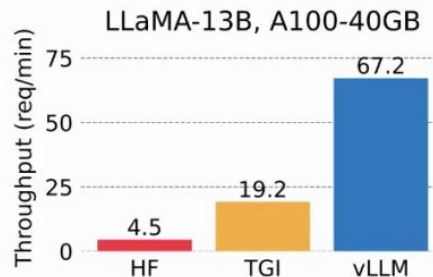
Пропускная способность

vLLM позволяет увеличить количество обрабатываемых запросов в минуту

- В сравнении с HF: в 14-24 раза
- В сравнении с TGI: в 2.2-2.5 раза

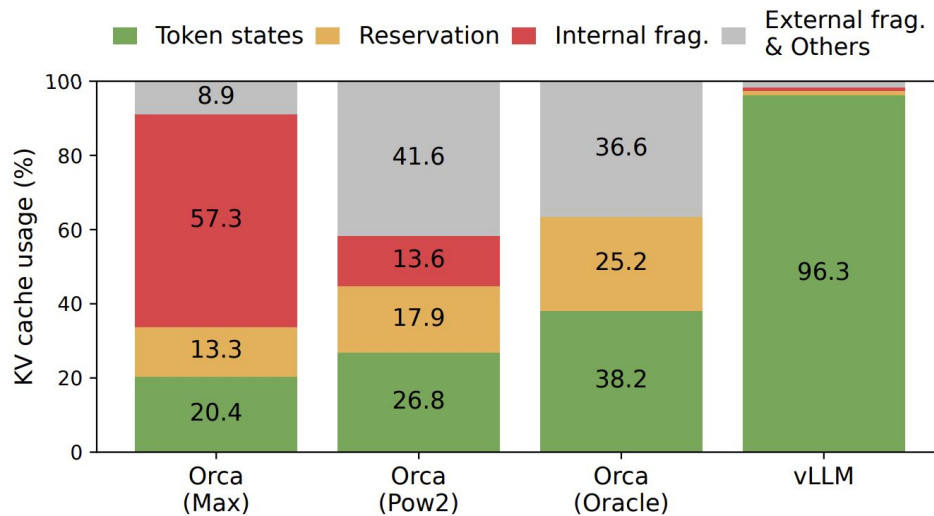


Serving throughput when each request asks for *one output completion*. vLLM achieves 14x - 24x higher throughput than HF and 2.2x - 2.5x higher throughput than TGI.



Итого

- Меньше GPU памяти → больше контекста и пользователей.
- Меньше копирований → быстрее обработка длинных запросов.
- Нет полной перезаписи KV-cache → стабильная скорость на длинных диалогах.
- Эффективно используем память под KV-cache
- Возможность переиспользования KV-cache



gguf и ray

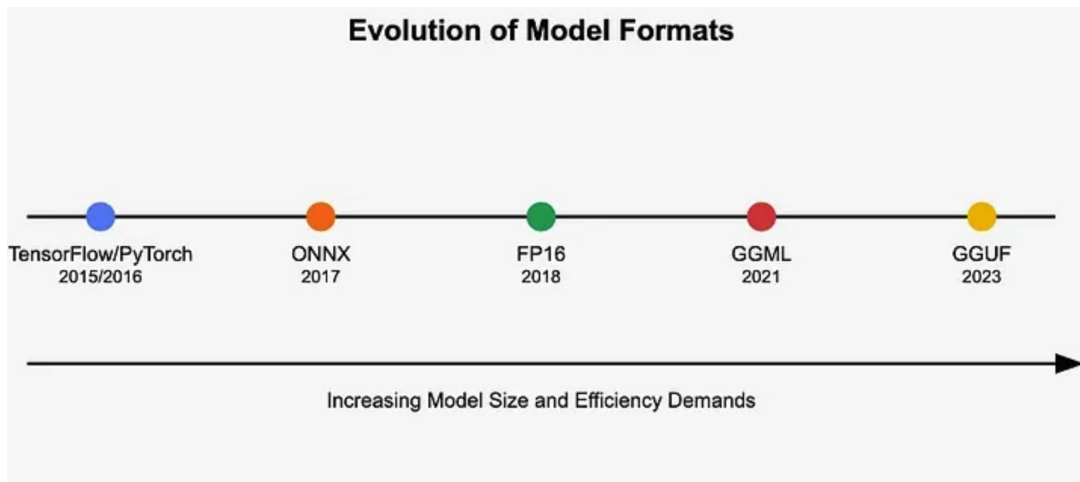


GGUF

(Generic GPT Unified Format)

GGUF - формат, созданный для эффективного хранения и запуска квантованных LLM.

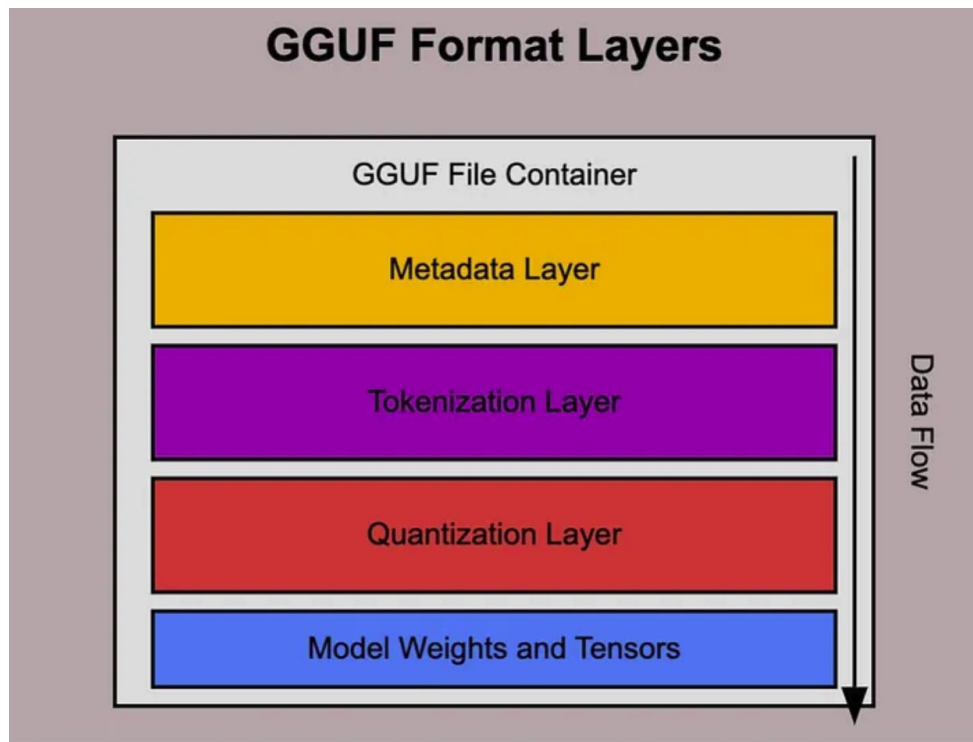
- Можно запускать на слабом железе
- Поддерживает самые разные архитектуры, размеры и квантизации
- Подходит даже для самых больших LLM



Что хранит GGUF?

GGUF позволяет сохранить в одном контейнере множество файлов.

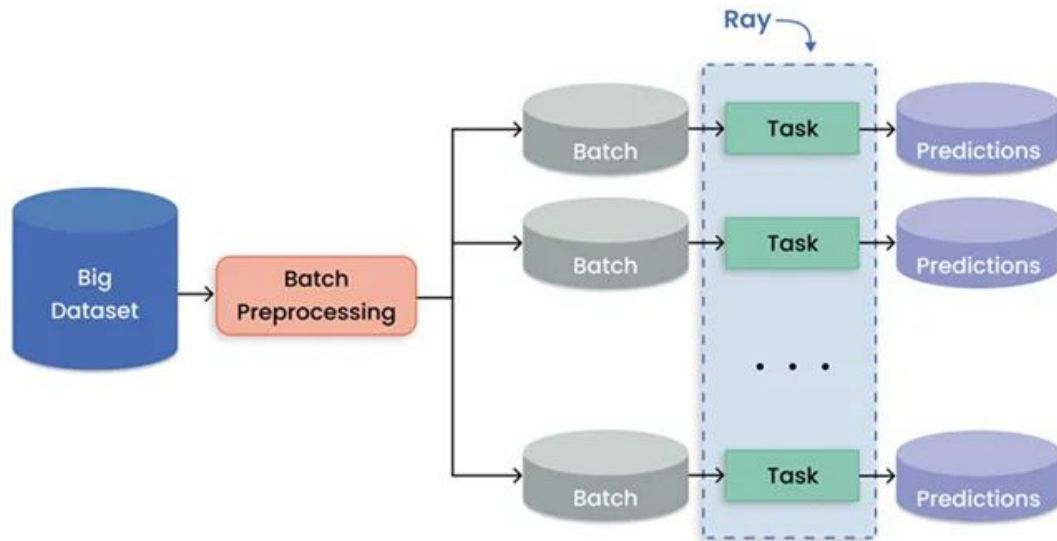
- Архитектура модели и её веса
- Токенизатор
- Гиперпараметры
- Используемый тип квантизации



Ray

Ray — фреймворк для распределённых вычислений, позволяет масштабировать Python-код от одного компьютера до кластера.

- Легко распараллелить python-код
- Ray Train ускоряет обучение модели, распределяя вычисления по CPU, GPU и кластерам.
- Поддержка состояния между процессами
 - можно шерить KV-кэши на инференсе!



Q&A

Спасибо
за внимание!

Ресурсы

- [vllm](#)
- [gguf](#)
- [ray](#)
- [LLama.cpp](#)
- [TGI](#)